

1. Mục tiêu bài thực hành

Sinh viên xây dựng chương trình chơi cờ Caro giữa người chơi và máy tính. Máy tính cần sử dụng các thuật toán tìm kiếm có đối thủ để lựa chọn nước đi. Bài thực hành giúp sinh viên vận dụng các nội dung sau:

- Biểu diễn trạng thái trò chơi và không gian trạng thái.
- Sinh nước đi hợp lệ và kiểm tra trạng thái kết thúc.
- Cài đặt thuật toán Minimax cho trò chơi hai người, tổng bằng không.
- Cải tiến Minimax bằng cắt nhánh Alpha-Beta.
- Sử dụng giới hạn độ sâu tìm kiếm và hàm đánh giá trạng thái.
- Đo đặc số trạng thái đã xét, thời gian chạy và phân tích hiệu quả thuật toán.

2. Mô tả bài toán

Trò chơi sử dụng là cờ Caro. Chương trình có thể chạy bằng console hoặc có giao diện đồ họa đơn giản.

- Bàn cờ có kích thước tối thiểu 9x9.
- Hai người chơi lần lượt đánh quân vào các ô trống.
- Có thể ký hiệu người chơi là X, máy là O, ô trống là dấu chấm (.).
- Người thắng là người có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.
- Không xét luật chặn hai đầu.
- Nếu bàn cờ đầy và không có người thắng thì kết quả là hòa.

3. Các mức thực hiện

a. Level 1 - Cài đặt AI chơi Caro bằng Minimax

Mục tiêu của Level 1 là xây dựng chương trình chơi Caro giữa người và máy, trong đó máy sử dụng thuật toán Minimax có giới hạn độ sâu để chọn nước đi.

Yêu cầu chức năng:

- Cài đặt bàn cờ kích thước tối thiểu 9x9.
- Cài đặt luật chơi: người chơi và máy đánh luân phiên, không đánh vào ô đã có quân.
- Trò chơi kết thúc khi một bên có 4 quân liên tiếp hoặc bàn cờ đầy.
- Cho phép người chơi đấu với máy.
- Máy tính chọn nước đi bằng thuật toán Minimax.
- Có giới hạn độ sâu tìm kiếm.
- Có hàm đánh giá trạng thái cho các trạng thái chưa kết thúc.
- Hiển thị nước đi do máy chọn.

- Ghi nhận nước đi được chọn, giá trị đánh giá, độ sâu tìm kiếm, số trạng thái đã xét và thời gian chạy.

Yêu cầu thuật toán:

- Nếu trạng thái là thắng/thua/hòa thì trả về giá trị kết thúc.
- Nếu đạt độ sâu giới hạn thì dùng hàm đánh giá trạng thái.
- Nếu là lượt máy thì chọn giá trị lớn nhất.
- Nếu là lượt người thì chọn giá trị nhỏ nhất.
- Thuật toán phải trả về nước đi tốt nhất và giá trị đánh giá tương ứng.

Gợi ý hàm đánh giá:

- Máy có 4 quân liên tiếp: điểm rất lớn.
- Người chơi có 4 quân liên tiếp: điểm rất nhỏ.
- Máy có 3 quân liên tiếp còn khả năng mở rộng: điểm cao.
- Người chơi có 3 quân liên tiếp: điểm âm lớn để ưu tiên chặn.
- Máy có 2 quân liên tiếp: điểm dương nhỏ.
- Người chơi có 2 quân liên tiếp: điểm âm nhỏ.

b. Level 2 - Cải tiến bằng Alpha-Beta pruning

Mục tiêu của Level 2 là phát triển tiếp từ Level 1 bằng cách cài đặt thêm thuật toán Alpha-Beta pruning để giảm số trạng thái cần xét.

Yêu cầu chức năng:

- Cài đặt thuật toán Alpha-Beta pruning.
- Alpha-Beta dùng cùng hàm đánh giá với Minimax.
- Alpha-Beta dùng cùng độ sâu tìm kiếm với Minimax khi so sánh.
- Có thể chọn chế độ AI: Minimax hoặc Alpha-Beta.
- Chạy hai thuật toán trên cùng một trạng thái bàn cờ.
- Ghi nhận nước đi được chọn, giá trị đánh giá, số trạng thái đã xét và thời gian chạy.

Yêu cầu thuật toán:

- α là giá trị tốt nhất hiện tại của người chơi MAX.
- β là giá trị tốt nhất hiện tại của người chơi MIN.
- Nếu $\beta \leq \alpha$ thì cắt nhánh.
- Thuật toán phải trả về nước đi tốt nhất và giá trị đánh giá tương ứng.
- Khi so sánh với Minimax, cần dùng cùng độ sâu và cùng hàm đánh giá để đảm bảo công bằng.

Gợi ý cải tiến hợp lệ:

- Chỉ sinh các nước đi gần những quân đã đánh.
- Ưu tiên xét các ô gần trung tâm bàn cờ.
- Ưu tiên xét các ô có khả năng tạo chuỗi 2, 3 hoặc 4 quân.
- Có thể sắp xếp nước đi trước khi mở rộng, nhưng phải ghi rõ trong báo cáo.

c. Level 3 - Phân tích, đánh giá và báo cáo thực nghiệm

Mục tiêu của Level 3 là đánh giá hiệu quả của Minimax và Alpha-Beta thông qua thực nghiệm trên nhiều trạng thái bàn cờ khác nhau.

Yêu cầu thực nghiệm:

- Chuẩn bị ít nhất 5 trạng thái bàn cờ khác nhau để kiểm thử.
- Chạy Minimax và Alpha-Beta trên cùng các trạng thái.
- Sử dụng cùng độ sâu tìm kiếm và cùng hàm đánh giá khi so sánh.
- Có thể thử thêm nhiều độ sâu khác nhau, ví dụ độ sâu 1, 2, 3.
- Lưu lại kết quả theo bảng để phân tích.

Các trạng thái kiểm thử gợi ý:

- Trạng thái đầu ván.
- Trạng thái giữa ván.
- Trạng thái máy có thể thắng ngay.
- Trạng thái người chơi sắp thắng, máy cần chặn.
- Trạng thái hai bên đều có cơ hội tấn công.
- Trạng thái có nhiều nước đi hợp lệ khiến thuật toán phải xét nhiều nhánh.

Nội dung phân tích bắt buộc:

- Alpha-Beta có chọn cùng nước đi với Minimax không?
- Alpha-Beta giảm được bao nhiêu trạng thái so với Minimax?
- Thời gian chạy thay đổi như thế nào khi tăng độ sâu?
- Độ sâu tìm kiếm ảnh hưởng thế nào đến chất lượng nước đi?
- Hàm đánh giá có ưu điểm gì và còn hạn chế gì?
- Trường hợp nào AI chơi tốt và trường hợp nào AI chọn nước đi chưa hợp lý?
- Nếu cải tiến tiếp, sinh viên sẽ cải tiến phần nào?

4. Tài nguyên GitHub mẫu được cung cấp

Sinh viên được cung cấp hai repository GitHub mẫu để tham khảo. Các tài nguyên này nhằm giúp sinh viên hiểu cách tổ chức chương trình và hình dung các thành phần của một game Caro/Gomoku AI, không phải lời giải để nộp lại.

Link 1: <https://github.com/husus/gomokuAI-py>

Link 2: https://github.com/MonHauVD/Caro_AI

4.1. Repo gomokuAI-py

Repo gomokuAI-py là một ví dụ nhỏ về Gomoku AI viết bằng Python. Dự án dùng bàn cờ 15x15, luật thắng 5 quân liên tiếp, AI sử dụng Minimax kết hợp Alpha-Beta pruning và giao diện được xây dựng bằng pygame.

Tài nguyên này có thể tham khảo:

- Cách tổ chức project Python cho game Gomoku/Caro.
- Cách tách phần giao diện, phần xử lý game và phần AI.
- Cách biểu diễn bàn cờ bằng mảng hai chiều.
- Cách kiểm tra nước đi hợp lệ và kiểm tra thắng/thua.
- Cách xây dựng giao diện đơn giản bằng pygame.
- Cách viết script đo thời gian chạy theo các độ sâu tìm kiếm khác nhau.

Lưu ý: repo này dùng luật Gomoku 5 quân liên tiếp, trong khi đề yêu cầu Caro 4 quân liên tiếp. Sinh viên không được sao chép nguyên trạng mà phải tự cài đặt và điều chỉnh luật chơi theo yêu cầu đề bài.

4.2. Repo Caro_AI

Repo Caro_AI là ví dụ lớn hơn về game Caro AI, có nhiều thành phần như game logic, giao diện, AI agent, cấu hình độ khó, benchmark và notebook phân tích kết quả.

Tài nguyên này có thể tham khảo:

- Cách tách chương trình thành các module game, UI, AI agent, config và benchmark.
- Cách tổ chức chế độ người chơi đấu với AI và AI đấu với AI.
- Cách cấu hình độ khó và tham số tìm kiếm.
- Cách ghi kết quả benchmark, lưu bàn cờ và log nước đi.
- Cách phân tích kết quả thực nghiệm bằng notebook.
- Một số ý tưởng nâng cao như move ordering, iterative deepening, time budget, transposition table và Zobrist hashing.

Lưu ý: các kỹ thuật nâng cao trong repo này chỉ là phần tham khảo. Sinh viên không bắt buộc phải cài đặt toàn bộ. Phần bắt buộc của đề là Minimax, Alpha-Beta, hàm đánh giá, đo đạc và phân tích kết quả.

5. Phạm vi sử dụng tài nguyên mẫu

a. Sinh viên được phép tham khảo

- Cách tổ chức thư mục source code và đặt tên file.
- Cách biểu diễn bàn cờ và xử lý lượt chơi.
- Cách kiểm tra nước đi hợp lệ, kiểm tra thắng/thua theo bốn hướng.
- Ý tưởng tổng quát của Minimax và Alpha-Beta pruning.
- Ý tưởng xây dựng hàm đánh giá trạng thái.
- Ý tưởng giới hạn danh sách nước đi cần xét để giảm không gian tìm kiếm.
- Cách đo thời gian chạy, ghi log và trình bày kết quả thực nghiệm.
- Cách viết README hướng dẫn cài đặt và chạy chương trình.

b. Sinh viên phải tự cài đặt

- Luật chơi Caro theo đúng yêu cầu đề bài.
- Cách biểu diễn trạng thái bàn cờ và sinh nước đi hợp lệ.
- Hàm kiểm tra trạng thái kết thúc.
- Thuật toán Minimax ở Level a.
- Thuật toán Alpha-Beta pruning ở Level b.
- Hàm đánh giá trạng thái bàn cờ.
- Cơ chế giới hạn độ sâu tìm kiếm.
- Cơ chế đếm số trạng thái đã xét và đo thời gian chạy.
- Bộ trạng thái kiểm thử và bảng kết quả thực nghiệm.
- Phân phân tích, nhận xét và báo cáo.

c. Sinh viên không được làm

- Không nộp nguyên trạng source code lấy từ GitHub.
- Không fork repo mẫu rồi chỉ đổi tên project hoặc đổi giao diện.
- Không sao chép nguyên thuật toán Minimax, Alpha-Beta hoặc hàm đánh giá mà không hiểu và không giải thích được.
- Không dùng thư viện hoặc mã nguồn có sẵn để thay thế phần thuật toán chính.
- Không nộp chương trình không chạy được.
- Không nộp báo cáo chỉ mô tả lý thuyết mà không có kết quả chạy thử.

6. Yêu cầu báo cáo

Báo cáo trình bày ngắn gọn nhưng phải có đủ thông tin để giảng viên hiểu cách cài đặt và đánh giá kết quả.

- 1 Mô tả bài toán cờ Caro.
- 2 Luật chơi và điều kiện thắng.
- 3 Cách biểu diễn trạng thái bàn cờ.
- 4 Cách sinh nước đi hợp lệ.
- 5 Cách kiểm tra trạng thái kết thúc.
- 6 Thuật toán Minimax đã cài đặt.
- 7 Thuật toán Alpha-Beta đã cài đặt.
- 8 Hàm đánh giá trạng thái.
- 9 Thiết kế các trạng thái thử nghiệm.
- 10 Bảng kết quả thực nghiệm.
- 11 Nhận xét về số trạng thái đã xét và thời gian chạy.
- 12 Nhận xét về ảnh hưởng của độ sâu tìm kiếm.
- 13 Ưu điểm và hạn chế của chương trình.
- 14 Link repository Github của nhóm
- 15 Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu.

7. Yêu cầu nộp bài

Hình thức nộp bài:

1. Nộp báo cáo pdf (mssv1_mssv2_mssv3.pdf)
2. Yêu cầu về Repository Github:
 - Đặt tên theo mẫu: mssv1_mssv2_mssv3_CaroAI
 - source_code/ (thư mục chứa mã nguồn)
 - requirements.txt
 - README.md (khoảng 1–2 trang, tập trung vào cách chạy chương trình)